

code_submission

Yuchen Chen

2026-03-08

Set up

Load data

```
all.aggWsex <- read_csv(paste0(data_location, "/data_submission.csv"))

## Rows: 248 Columns: 9
## -- Column specification -----
## Delimiter: ","
## chr (8): groupID, actor, subject, actor.sex, subject.sex, dyad.sex1, dyad.se...
## dbl (1): nevents
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.

bird.list <- read_csv(paste0(data_location, "/bird_submission.csv"))

## Rows: 50 Columns: 5
## -- Column specification -----
## Delimiter: ","
## chr (5): source, combogroupID, Band No, Sex, Color
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Separate to different groups

```
split_dfs <- split(all.aggWsex, all.aggWsex$groupID)
names(split_dfs)

## [1] "G1" "G2" "G3" "G4" "G5"

#Calculate rank
```

Load springrank code from Cate/Dan

```
library(igraph)
library(Matrix)
library(Rlinsolve)

## ** ----- **

## ** Rlinsolve

## ** - Solving (Sparse) System of Linear Equations

## **

## ** Version      : 0.3.3      (2026)

## ** Maintainer  : Kisung You (kisung.you@outlook.com)

## **

## ** Please share any bugs or suggestions to the maintainer.

## ** ----- **

spring_rank <- function(A, alpha = 0, l0 = 1.0, l1 = 1.0,
                        shift = TRUE, solver = Rlinsolve::lsolve.bicgstab) {
  #' Core function for calculating SpringRank.
  #' Default parameters follow stanadard model.
  #'
  #' @param A The adjacency matrix of the graph. This should be a sparse dgCMatrix;
  #' if it isn't it will be coerced to a dgCMatrix.
  #' @param alpha Controls the impact regularization term.
  #' @param l0 Regularization spring's rest length.
  #' @param l1 Interaction springs' rest length.
  #' @param shift (Optional, default TRUE) normalize such that the lowest-ranked
  #' node has a SpringRank value of zero.
  #' @param solver (Optional, default Rlinsolve::bicgstab) your preferred
  #' solver for  $Ax=B$ . Should be able to handle dgCMatrix sparse matrices.
  #' if the solve is not from Rlinsolve may throw a spurious warning for
  #' unused parameters.
  #'
  #' @return A vector of SpringRank scores for each node. Sort or order the
  #' vector for ordinal rankings of each node.

  if (class(A) == "matrix") {
    # coerce dense matrix to sparse matrix so the user doesn't have to.
    A <- as(Matrix(A, sparse = TRUE, doDiag = FALSE), "dgCMatrix")
  } else {
    # confirm it's the right kind of sparse matrix. might throw an error
    # if it's one of the less common sparse matrix types.
  }
```

```

  A <- as(A, "dgCMatrix")
}

N <- dim(A)[1]
k_in <- colSums(A)
k_out <- rowSums(A)
One <- as(Matrix(rep(1, N)), "dgCMatrix")
C <- A + t(A)

D1 <- as(Matrix(0, ncol = N, nrow = N), "dgCMatrix")
D2 <- as(Matrix(0, ncol = N, nrow = N), "dgCMatrix")

for (i in 1:N) {
  D1[i, i] <- k_out[i] + k_in[i]
  D2[i, i] <- l1 * (k_out[i] - k_in[i])
}

if (alpha != 0) {
  print("assuming invertible matrix")

  B = One * l0 + D2 %*% One
  A_ = alpha * diag(nrow = N, ncol = N) + D1 - C
} else {
  print("fixing a rank degree of freedom")

  C <- C +
    matrix(rep(A[N,] , times = N),
           ncol = N,
           nrow = N,
           byrow = T) +
    matrix(rep(A[, N], times = N),
           ncol = N,
           nrow = N,
           byrow = T)

  D3 <- as(Matrix(0, ncol = N, nrow = N), "dgCMatrix")
  for (i in 1:N) {
    D3[i, i] <- l1 * (k_out[N] - k_in[N])
  }

  B <- D2 %*% One + D3 %*% One
  A_ <- D1 - C
}

rank <- solver(A_, B, verbose = F)
if (class(rank) == "list") {
  rank <- rank$x # accomodates both Rlinsolve and Matrix solves
}

if (shift) {
  rank <- rank - min(rank)
}

```

```

# coerce matrix to vector, so we can use names()
rank <- rank[,1]
names(rank) <- colnames(A)
return(rank)
}

spring_rank_network <- function(N, beta, alpha, K, l0 = 0.5, l1 = 1.0) {
  #' Generative model for SpringRank. Builds a weighted graph with self-loops.
  #' The model first generates node scores using a normal distribution, then
  #' generates edges (and edge weights) from a Poisson distribution with mean
  #' equal to the energy of the system.
  #'
  #' @param N The number of nodes.
  #' @param beta Inverse temperature, a noise parameter.
  #' @param alpha Variance of the Normal prior.
  #' @param K The average degree of the network.
  #' @param l0 Prior spring's rest length.
  #' @param l1 Interaction spring's rest length.
  #'
  #' @return A directed graph (potentially weighted, potentially containing
  #' self-loops)

  scores <- rnorm(N, l0, 1/sqrt(alpha*beta))
  Z <- 0
  for (i in 1:N) {
    for (j in 1:N) {
      Z <- Z + exp(-0.5 * beta * (scores[i] - scores[j] - l1)^2)
    }
  }
  C <- (K*N)/Z

  # for loops are slow in R so make a matrix of element-wise subtractions,
  # each element i, j being scores[i]-scores[j]
  # basically, trading off increased memory usage (dense matrix) for speed.
  scores_mat <- matrix(1, length(scores), 1) %*% t(scores)
  scores_mat <- scores_mat - scores - l1
  H <- .5 * scores_mat^2
  lambda <- C * exp(-1*beta*H)
  A <- rpois(length(scores)^2, lambda) %>% matrix(nrow = dim(lambda)[1])

  return(graph_from_adjacency_matrix(A, mode = "directed", weighted = "weight"))
}

```

Calculate springrank power, ordinal rank, rule breaking etc for each group

```

compute_springrank <- function(df) {

  agg.el <- df %>% ungroup()

  agg.el <- agg.el %>% select(actor, subject, subject.sex, actor.sex, nevents)

```

```

g <- graph.data.frame(agg.el)
g.adj <- get.adjacency(g, sparse = TRUE, attr='nevents')
rank_spring <- as.data.frame(spring_rank(g.adj))

rank_spring$birdID <- rownames(rank_spring)
colnames(rank_spring) <- c("springrank_value", "birdID")
rank_spring <- rank_spring %>% arrange(desc(springrank_value))
rank_spring$rank_springrank <- seq(1:nrow(rank_spring))

rank_subjects <- rank_spring %>%
  rename(
    subject.power = springrank_value,
    subject = birdID,
    subject.rank = rank_springrank
  )

rank_actors <- rank_spring %>%
  rename(
    actor.power = springrank_value,
    actor = birdID,
    actor.rank = rank_springrank
  )

df <- df %>%
  left_join(rank_subjects, by = "subject") %>%
  left_join(rank_actors, by = "actor")

df <- df %>%
  mutate(power.diff = actor.power - subject.power,
         rank.diff = actor.rank - subject.rank)
}

split_dfs_Wrank <- map(split_dfs, compute_springrank)

## Warning: 'graph.data.frame()' was deprecated in igraph 2.0.0.
## i Please use 'graph_from_data_frame()' instead.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

## Warning: 'get.adjacency()' was deprecated in igraph 2.0.0.
## i Please use 'as_adjacency_matrix()' instead.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

## 'as(<dgeMatrix>, "dgCMatrix")' is deprecated.
## Use 'as(., "CsparseMatrix")' instead.
## See help("Deprecated") and help("Matrix-deprecated").

```

```
## [1] "fixing a rank degree of freedom"
## [1] "fixing a rank degree of freedom"
## [1] "fixing a rank degree of freedom"
## [1] "fixing a rank degree of freedom"
## [1] "fixing a rank degree of freedom"
```

```
G1 <- compute_springrank(split_dfs[["G1"]])
```

```
## [1] "fixing a rank degree of freedom"
```

```
G2 <- compute_springrank(split_dfs[["G2"]])
```

```
## [1] "fixing a rank degree of freedom"
```

```
G3 <- compute_springrank(split_dfs[["G3"]])
```

```
## [1] "fixing a rank degree of freedom"
```

```
G4 <- compute_springrank(split_dfs[["G4"]])
```

```
## [1] "fixing a rank degree of freedom"
```

```
G5 <- compute_springrank(split_dfs[["G5"]])
```

```
## [1] "fixing a rank degree of freedom"
```

Data cleaning

datasets that we are using:

rank_df_all : 50 rows of birds with power and sex

combined_df_Wrank_sexb : 248 rows of dyadic interactions with power and sex

```
combined_df_Wrank_sexb <- bind_rows(
  "G1" = G1,
  "G2" = G2,
  "G3" = G3,
  "G4" = G4,
  "G5" = G5,
  .id = "groupID"
)

combined_df_Wrank_sexb %>%
  group_by(groupID) %>%
  summarise(sumagg = sum(nevents)) %>%
  ungroup()
```

```
## # A tibble: 5 x 2
##   groupID sumagg
##   <chr>    <dbl>
## 1 G1      2531
## 2 G2      1404
## 3 G3       831
## 4 G4      1073
## 5 G5       345
```

```
df1_all <- combined_df_Wrank_sexb %>%
  distinct(birdID = actor, rank = actor.rank, power = actor.power, groupID = groupID, sex = actor.sex)

df2_all <- combined_df_Wrank_sexb %>%
  distinct(birdID = subject, rank = subject.rank, power = subject.power, groupID = groupID, sex = subject.sex)

rank_df_all <- bind_rows(df1_all, df2_all) %>%
  distinct(birdID, .keep_all = TRUE)
```

#1. Individual Level

##Plotting

Plot - 5 groups

agg_by_actor_group : 50 rows of birds with total aggression events, sex, and power

```
colrs <- c("Female" = "#F8766D", "Male" = "#00BFC4")

#Aggregate nevents per actor (bird) within each group
agg_by_actor_group <- combined_df_Wrank_sexb %>%
  group_by(groupID, actor, actor.sex) %>%
  summarise(total_events = sum(nevents), .groups = "drop")

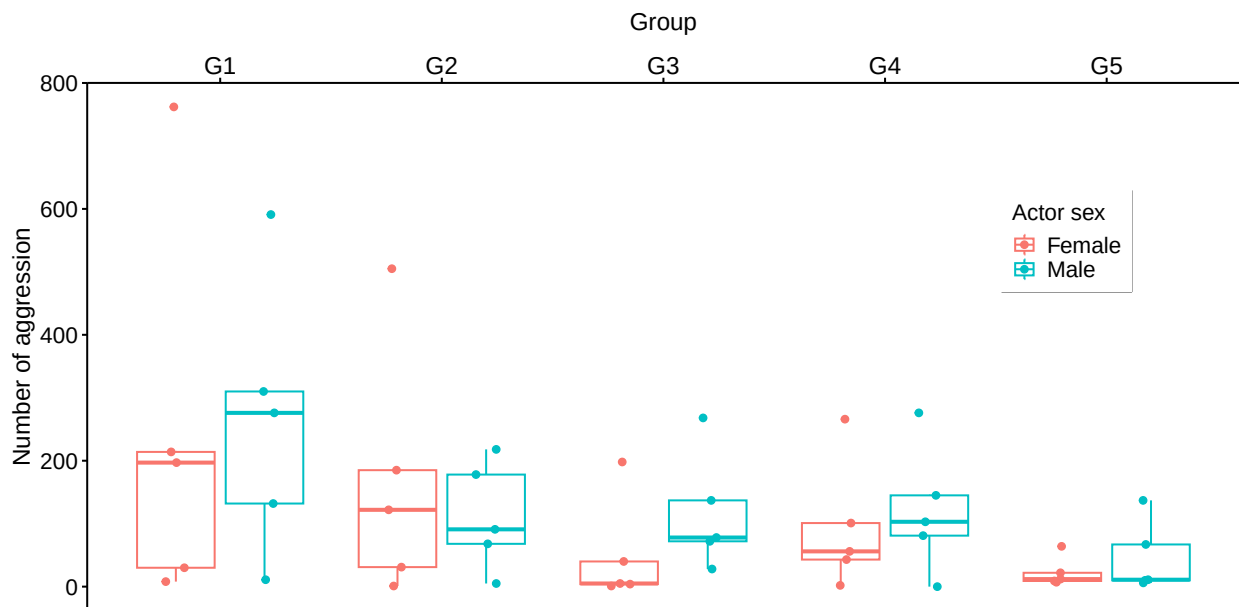
#There is a bird never becomes aggressor
agg_by_actor_group <- rank_df_all %>%
  left_join(agg_by_actor_group, join_by("birdID"=="actor") ) %>%
  mutate(groupID = coalesce(groupID.x, groupID.y),
         sex = coalesce(sex, actor.sex)) %>%
  select(birdID, rank, power, groupID, sex, total_events) %>%
  mutate(total_events = if_else(is.na(total_events), 0, total_events))

#Boxplot by group and sex showing total aggression per bird
sex_boxplot_group <- ggboxplot(agg_by_actor_group, x = "groupID", y = "total_events", color = "sex", add = TRUE) +
  scale_color_manual(values = colrs) +
  #theme_minimal() +
  labs(y = "Number of aggression", color = "Actor sex", x = "Group") +
  scale_x_discrete(position = "top",
                  labels = c("G1", "G2", "G3", "G4", "G5") )+
  theme(legend.position = "bottom",
        axis.title.y = element_text(size = 14),
        axis.text.y = element_text(size = 13),
        axis.title.x = element_text(size = 14),
        axis.title.x.top = element_text(size = 14, margin = margin(b = 12)),
        axis.text.x = element_text(size = 14),
        legend.text = element_text(size = 13),
```

```

    legend.title = element_text(size = 13))+
  theme(legend.position = c(0.85, 0.7),
    legend.box.background = element_rect(color = "black", fill = "white", linewidth = 0.2),
    legend.key.size = unit(0.5, 'cm'))+
  theme(
    axis.line.y.left = element_line(color = "black"),
    axis.line.x.top = element_line(color = "black"),
    axis.ticks = element_line(color = "black"),
    axis.line = element_blank()
  )
sex_boxplot_group

```



```

ggsave(file.path(figure_output_folder, "sex_boxplot_group.jpeg"),
  sex_boxplot_group,
  device = "jpeg",
  width = 15, height = 10, units = "cm")

```

Plot - all together

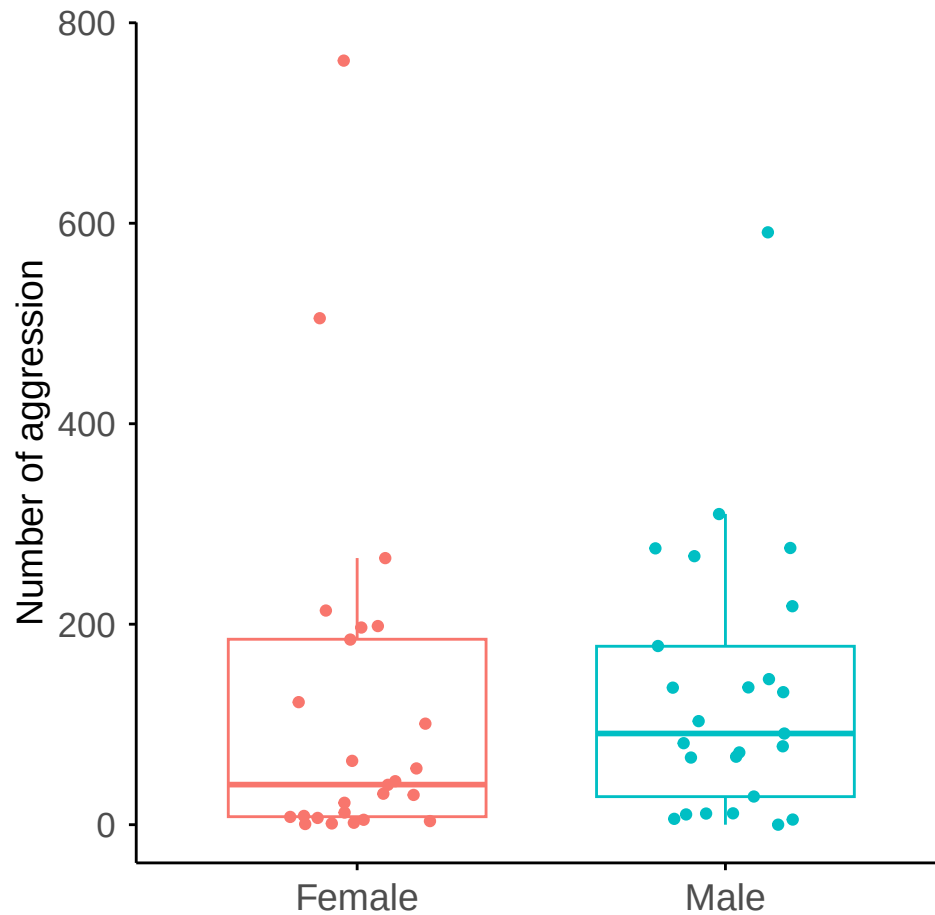
```

#Boxplot all groups combined, by sex, total aggression per bird
sex_boxplot_all <- ggboxplot(agg_by_actor_group, x = "sex", y = "total_events", color = "sex", add = "jitter")
  scale_color_manual(values = colrs) +
  theme_minimal() +
  theme(legend.position = "none",
    axis.text.x = element_text(size = 14),
    axis.title.y = element_text(size = 14),
    axis.text.y = element_text(size = 13))+
  labs(y = "Number of aggression", x = "") +
  theme(
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank()
  )

```



```
)+
  theme(
    axis.line.y.left = element_line(color = "black"),
    axis.line.x.bottom = element_line(color = "black"),
    axis.ticks = element_line(color = "black"),
    axis.line = element_blank()
  )
sex_boxplot_all
```



```
ggsave(file.path(figure_output_folder, "sex_boxplot_all.jpeg"),
  sex_boxplot_all,
  device = "jpeg",
  width = 10, height = 10, units = "cm")
```

Get proportions within group

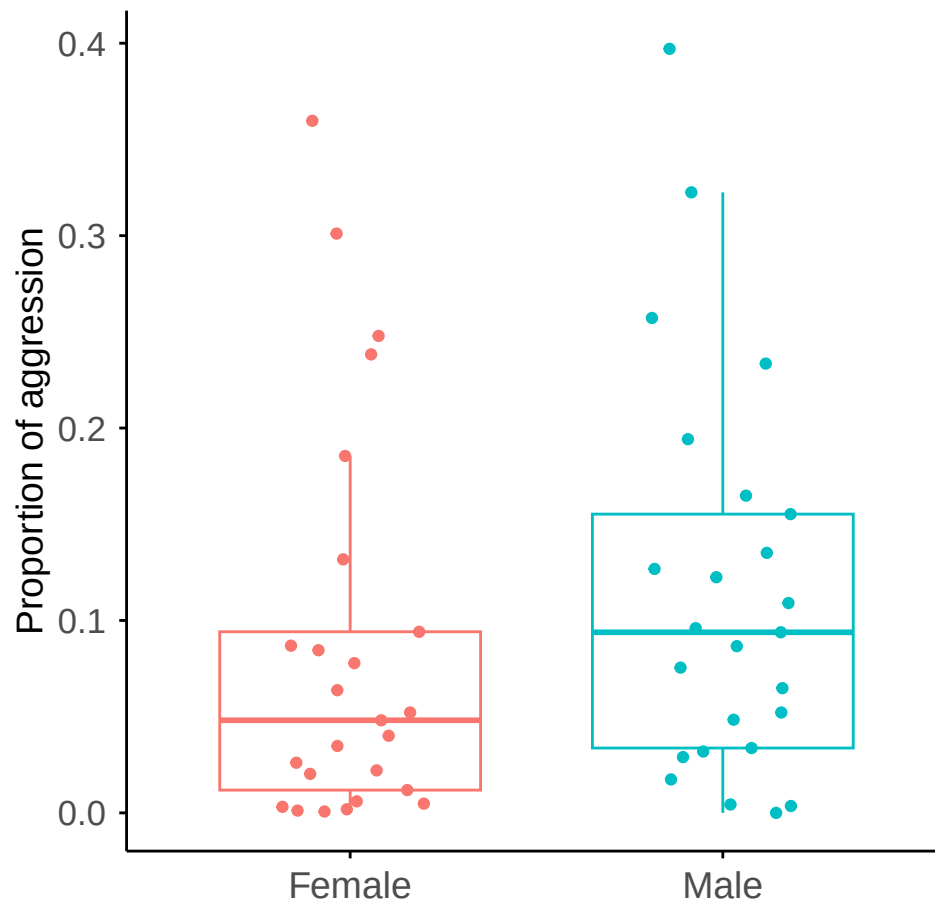
agg_by_actor_group : 50 rows of birds with total aggression events, proportions of aggression, sex, and power

```
### add group proportion
agg_by_actor_group <- agg_by_actor_group %>%
  group_by(groupID) %>%
  mutate(prop_agg = total_events/sum(total_events))
```

Plot 5 group within-group proportion

```
#Boxplot all groups combined, by sex, total aggression per bird
sex_boxplot_all_prop <- ggboxplot(agg_by_actor_group, x = "sex", y = "prop_agg", color = "sex", add = ".")
  scale_color_manual(values = colrs) +
  theme_minimal() +
  theme(legend.position = "none",
        axis.text.x = element_text(size = 14),
        axis.title.y = element_text(size = 14),
        axis.text.y = element_text(size = 13)) +
  labs(y = "Proportion of aggression", x = "") +
  theme(
    panel.grid.major = element_blank() ,
    panel.grid.minor = element_blank()
  ) +
  theme(
    axis.line.y.left = element_line(color = "black"),
    axis.line.x.bottom = element_line(color = "black"),
    axis.ticks = element_line(color = "black"),
    axis.line = element_blank()
  )

sex_boxplot_all_prop
```

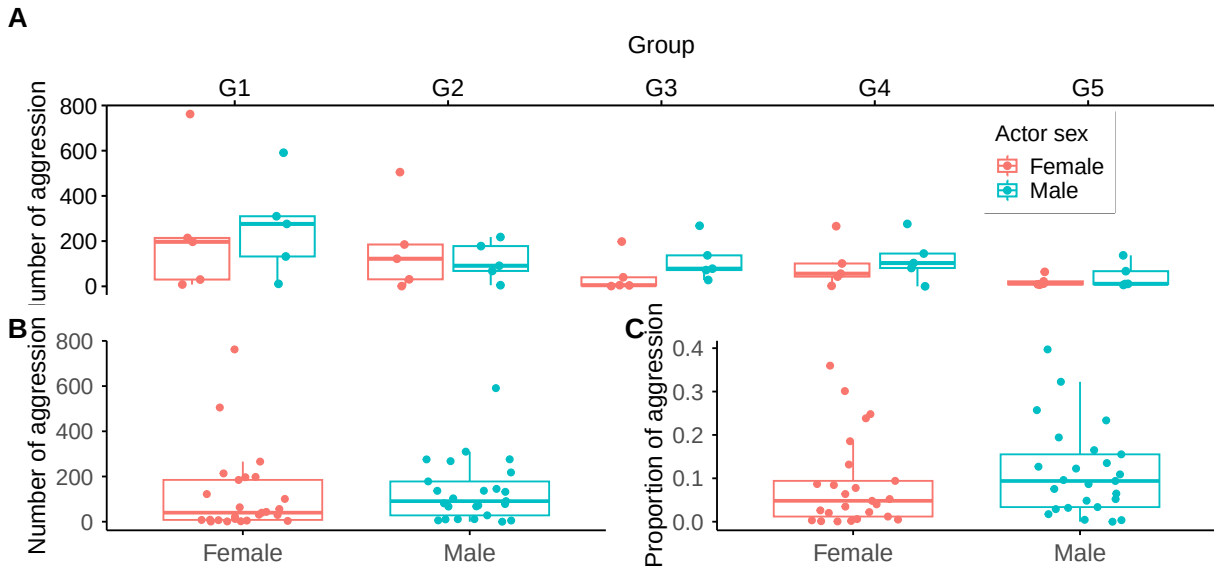


```
ggsave(file.path(figure_output_folder, "sex_boxplot_all_prop.jpeg"),
  sex_boxplot_all_prop,
  device = "jpeg",
  width = 10, height = 10, units = "cm")
```

Combine three individual level plots

```
sex_boxplot_combined_complete <- sex_boxplot_group /
  (sex_boxplot_all | sex_boxplot_all_prop) +
  plot_annotation(tag_levels = "A") &
  theme(plot.tag = element_text(face = "bold", size = 16))

sex_boxplot_combined_complete
```



```
ggsave(file.path(figure_output_folder, "sex_boxplot_combined_complete.tiff"),
        sex_boxplot_combined_complete,
        device = "tiff",
        width = 18, height = 16, units = "cm")
```

Analysis

5 groups separate T test

```
split_agg <- split(agg_by_actor_group, agg_by_actor_group$groupID)

# check before fitting
normality_test <- map(split_agg, ~ shapiro.test(.x$total_events))

variance_test <- map(split_agg, ~ leveneTest(total_events ~ sex, data = .x))

## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.
## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.
## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.
## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.
## Warning in leveneTest.default(y = y, group = group, ...): group coerced to
## factor.

normality_test_table <- map_df(normality_test, tidy, .id = "groupID")

normality_test_table <- normality_test_table %>%
  mutate(across(where(is.numeric), ~ signif(.x, 2)),
```

```
groupID = c("G1", "G2", "G3", "G4", "G5")) %>% select(-method)
```

```
# save the check
```

```
normality_test_table
```

```
## # A tibble: 5 x 3
##   groupID statistic p.value
##   <chr>      <dbl>   <dbl>
## 1 G1         0.87  0.11
## 2 G2         0.83  0.031
## 3 G3         0.86  0.072
## 4 G4         0.88  0.13
## 5 G5         0.71  0.0012
```

```
set_flextable_defaults(
  font.size = 10, theme_fun = theme_vanilla,
  padding = 6)
```

```
flextable(normality_test_table) %>%
  save_as_docx( path = paste0(table_output_folder, "/normality_test_table.docx"))
```

```
# fit the model
```

```
models_sep <- map(split_agg, ~ wilcox.test(total_events ~ sex, data = .x))
```

```
wtest_individual_sep <- map_df(models_sep, tidy, .id = "groupID")
```

```
wtest_individual_sep <- wtest_individual_sep %>%
  mutate(across(where(is.numeric), ~ signif(.x, 2)),
    groupID = c("G1", "G2", "G3", "G4", "G5")) %>% select(-method, -alternative)
```

```
wtest_individual_sep
```

```
## # A tibble: 5 x 3
##   groupID statistic p.value
##   <chr>      <dbl>   <dbl>
## 1 G1         10    0.69
## 2 G2         13     1
## 3 G3          5    0.15
## 4 G4          9    0.55
## 5 G5         11    0.84
```

```
# save the model result
```

```
flextable(wtest_individual_sep) %>%
  save_as_docx( path = paste0(table_output_folder, "/wtest_individual_sep.docx"))
```

```
write_csv(wtest_individual_sep, file = file.path(table_output_folder, "wtest_individual_sep.csv"))
```

5 groups all together GLMM

```
#with count data, negative binomial
```

```
individual_model_all_nb1 <- glmmTMB(total_events ~ sex + (1 | groupID),
```

```

        family = nbinom1, data = agg_by_actor_group)

individual_model_all_nb2 <- glmmTMB(total_events ~ sex + (1 | groupID),
        family = nbinom2, data = agg_by_actor_group)

AIC(individual_model_all_nb1 , individual_model_all_nb2 ) # not a big differences

##                df      AIC
## individual_model_all_nb1  4 579.1031
## individual_model_all_nb2  4 577.5532

aic_table <- as.data.frame(AIC(individual_model_all_nb1, individual_model_all_nb2))
flextable(aic_table) %>%
  save_as_docx( path = paste0(table_output_folder, "/AIC_individual_model_comparison.docx"))

#individual_model_all_nb2 is slightly a better fit

summary(individual_model_all_nb1)

## Family: nbinom1 ( log )
## Formula:          total_events ~ sex + (1 | groupID)
## Data: agg_by_actor_group
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    579.1    586.8   -285.6    571.1        46
##
## Random effects:
##
## Conditional model:
##   Groups  Name      Variance Std.Dev.
## groupID (Intercept) 0.01277  0.113
## Number of obs: 50, groups: groupID, 5
##
## Dispersion parameter for nbinom1 family (): 195
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   4.6571    0.2374  19.619  <2e-16 ***
## sexMale       0.2879    0.2518   1.143   0.253
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

summary(individual_model_all_nb2)

## Family: nbinom2 ( log )
## Formula:          total_events ~ sex + (1 | groupID)
## Data: agg_by_actor_group
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    577.6    585.2   -284.8    569.6        46
##
## Random effects:

```

```
##
## Conditional model:
##   Groups   Name      Variance Std.Dev.
##   groupID (Intercept) 0.2778   0.527
## Number of obs: 50, groups:  groupID, 5
##
## Dispersion parameter for nbinom2 family (:): 0.679
##
## Conditional model:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)  4.5768     0.3485  13.134 <2e-16 ***
## sexMale      0.2282     0.3527   0.647  0.518
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
r2(individual_model_all_nb1)
```

```
## # R2 for Mixed Models
##
##   Conditional R2: 0.721
##   Marginal R2: 0.450
```

```
r2(individual_model_all_nb2)
```

```
## # R2 for Mixed Models
##
##   Conditional R2: 0.243
##   Marginal R2: 0.011
```

```
individual_model_all_nb1_summary <- summary(individual_model_all_nb1)
individual_model_all_nb2_summary <- summary(individual_model_all_nb2)
```

```
# save the model results
```

```
individual_model_all_nb1_summary <- capture.output(summary(individual_model_all_nb1))
writeLines(individual_model_all_nb1_summary, file.path(table_output_folder, "individual_model_all_nb1_s
```

```
doc <- read_docx()
for(line in individual_model_all_nb1_summary){doc <- body_add_par(doc, line, style = "Normal")}
print(doc,target = file.path(table_output_folder,"individual_model_all_nb1_summary.docx"))
```

```
summary(individual_model_all_nb1)$coefficients$cond %>%
  as.data.frame() %>%
  tibble::rownames_to_column("Predictor") %>%
  flextable() %>%
  colformat_double(digits = 3) %>%
  add_footer_lines(
    "Estimate = log-scale coefficient; Std. Error = standard error; z value = Wald z statistic; Pr(>|z|)
  ) %>%
  autofit() %>%
  save_as_docx(
    path = file.path(table_output_folder, "individual_model_all_nb1_estimates.docx")
```

```

)

individual_model_all_nb2_summary <- capture.output(summary(individual_model_all_nb2))
writeLines(individual_model_all_nb2_summary, file.path(table_output_folder, "individual_model_all_nb2_s

doc <- read_docx()
for(line in individual_model_all_nb2_summary){doc <- body_add_par(doc, line, style = "Normal")}
print(doc,target = file.path(table_output_folder,"individual_model_all_nb2_summary.docx"))

summary(individual_model_all_nb2)$coefficients$cond %>%
  as.data.frame() %>%
  tibble::rownames_to_column("Predictor") %>%
  flextable() %>%
  colformat_double(digits = 3) %>%
  add_footer_lines(
    "Estimate = log-scale coefficient; Std. Error = standard error; z value = Wald z statistic; Pr(>|z|) = "
  ) %>%
  autofit() %>%
  save_as_docx(
    path = file.path(table_output_folder, "individual_model_all_nb2_estimates.docx")
  )

#check the model
individual_model_all_nb1_diagnostics_plot <- check_model(individual_model_all_nb1, title_size = 8, base

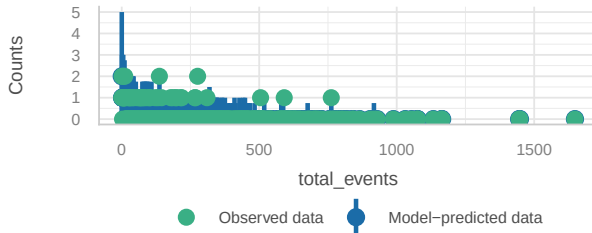
## 'check_outliers()' does not yet support models of class 'glmmTMB'.

individual_model_all_nb1_diagnostics_plot

```

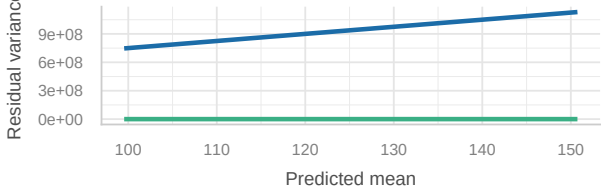

Posterior Predictive Check

Model-predicted intervals should include observed data points



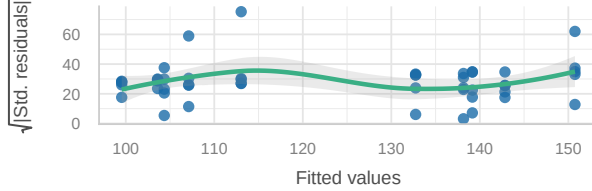
Misspecified dispersion and zero-inflation

Observed residual variance (green) should follow predicted residual variance



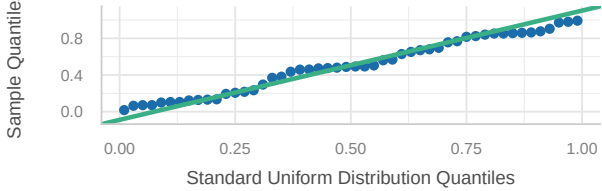
Homogeneity of Variance

Reference line should be flat and horizontal



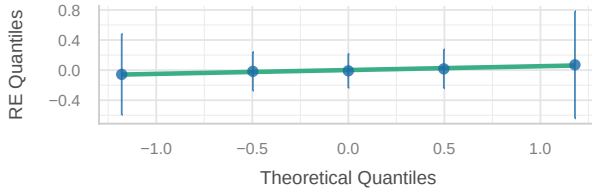
Distribution of Quantile Residuals

Dots should fall along the line



Normality of Random Effects (groupID)

Dots should be plotted along the line



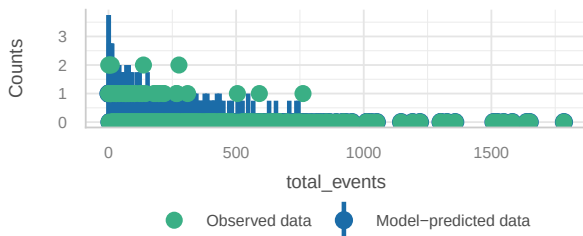
```
individual_model_all_nb2_diagnostics_plot <- check_model(individual_model_all_nb2, title_size = 8, base
```

```
## 'check_outliers()' does not yet support models of class 'glmmTMB'.
```

```
individual_model_all_nb2_diagnostics_plot
```

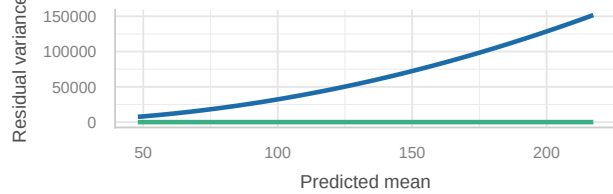
Posterior Predictive Check

Model-predicted intervals should include observed data points



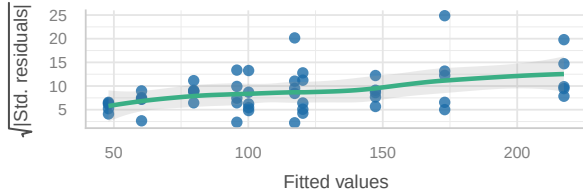
Misspecified dispersion and zero-inflation

Observed residual variance (green) should follow predicted residual variance



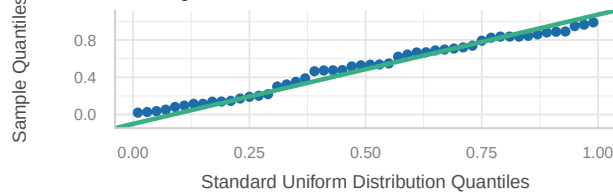
Homogeneity of Variance

Reference line should be flat and horizontal



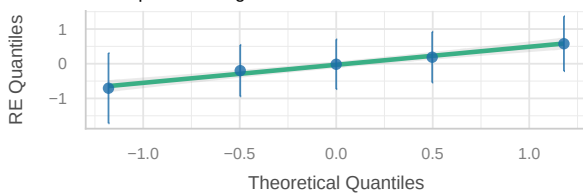
Distribution of Quantile Residuals

Dots should fall along the line



Normality of Random Effects (groupID)

Dots should be plotted along the line



```
# saving the plot
```

```
tiff(file = file.path(figure_output_folder, "individual_model_all_nb1_diagnostics.tiff"),
     width = 18, height = 18, units = "cm", res = 300)
```

```
print(individual_model_all_nb1_diagnostics_plot)
```

```
dev.off()
```

```
## pdf
```

```
## 2
```

```
tiff(file = file.path(figure_output_folder, "individual_model_all_nb2_diagnostics.tiff"),
     width = 18, height = 18, units = "cm", res = 300)
```

```
print(individual_model_all_nb2_diagnostics_plot)
```

```
dev.off()
```

```
## pdf
```

```
## 2
```

```
#with proportion data, beta regression
```

```
individual_model_all_beta_complete <- glmmTMB(
```

```
prop_agg ~ sex ,#+ (1 | groupID),
zi = ~ 1,
family = beta_family(),
data = agg_by_actor_group
)

summary(individual_model_all_beta_complete)
```

```
## Family: beta ( logit )
## Formula:          prop_agg ~ sex
## Zero inflation:    ~1
## Data: agg_by_actor_group
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##   -114.6   -106.9     61.3   -122.6      46
##
##
## Dispersion parameter for beta family (): 7.91
##
## Conditional model:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -2.4466    0.2156 -11.346  <2e-16 ***
## sexMale       0.4917    0.2560   1.921   0.0547 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -3.892     1.010  -3.853 0.000117 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
r2(individual_model_all_beta_complete)
```

```
## # R2 for Zero-Inflated and Hurdle Regression
##      R2: 0.000
##   adj. R2: -0.042
```

```
# save the model results
```

```
individual_model_all_beta_complete_summary <- capture.output(summary(individual_model_all_beta_complete))
writeLines(individual_model_all_beta_complete_summary, file.path(table_output_folder, "individual_model_all_beta_complete_summary.txt"))

doc <- read_docx()
for(line in individual_model_all_beta_complete_summary){doc <- body_add_par(doc, line, style = "Normal")}
print(doc,target = file.path(table_output_folder,"individual_model_all_beta_complete_summary.docx"))

summary(individual_model_all_beta_complete)$coefficients$cond %>%
  as.data.frame() %>%
  tibble::rownames_to_column("Predictor") %>%
  flextable() %>%
  colformat_double(digits = 3) %>%
```

```

add_footer_lines(
  "Estimate = logit-scale coefficient; Std. Error = standard error; z value = Wald z statistic; Pr(>|z|) = two-tailed p-value"
) %>%
autofit() %>%
save_as_docx(
  path = file.path(table_output_folder, "individual_model_all_beta_complete_estimates.docx")
)

```

#check the model

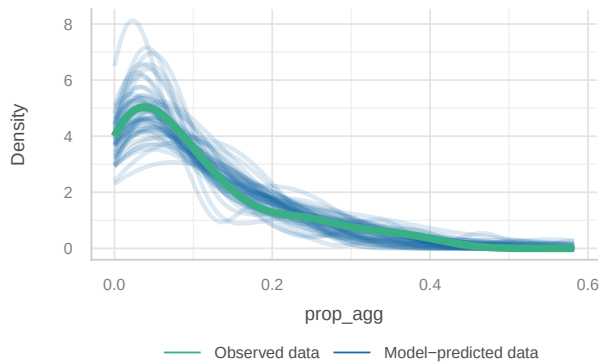
```
individual_model_all_beta_complete_diagnostics_plot <- check_model(individual_model_all_beta_complete, "
```

'check_outliers()' does not yet support models of class 'glmmTMB'.

```
individual_model_all_beta_complete_diagnostics_plot
```

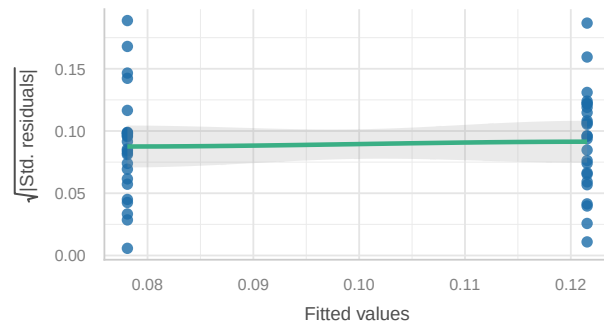
Posterior Predictive Check

Model-predicted lines should resemble observed data line



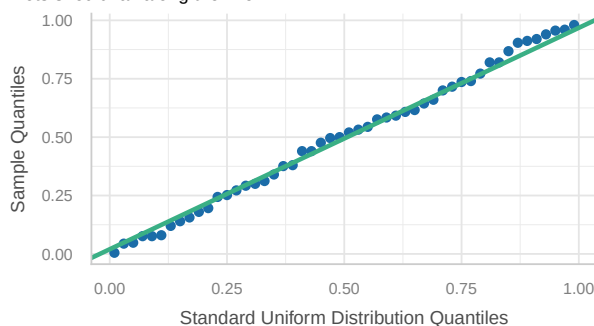
Homogeneity of Variance

Reference line should be flat and horizontal



Distribution of Quantile Residuals

Dots should fall along the line



saving the plot

```
tiff(file = file.path(figure_output_folder, "individual_model_all_beta_complete_diagnostics.tiff"),
     width = 18, height = 18, units = "cm", res = 300)
```

```
print(individual_model_all_beta_complete_diagnostics_plot)
```

```
dev.off()
```

pdf

2

#2.Dyad Level

Permutation

```
colrs3 <- c(
  "same sex"="chocolate",
  "opposite sex"="dimgray")
```

##2 dyad types

2 dyad type permutation test

```
set.seed(123)

sex.list <- c("Female", "Female", "Female", "Female", "Female",
             "Male", "Male", "Male", "Male", "Male")

sex.list <- as.data.frame(sex.list)

dyad_perm <- function(df, group_name){

  observed <- df %>%
    group_by(dyad.sex3) %>%
    summarise(mean_events = mean(nevents), .groups = "drop")

  t <- 5000
  perm_results <- matrix(0, nrow = t, ncol = 2)
  colnames(perm_results) <- c("same sex", "opposite sex")

  for (i in 1:t) {
    rows <- sample(nrow(sex.list))
    sex.list.group <- sex.list[rows, ]

    bird.list.group <- bird.list %>%
      filter(combogroupID == unique(df$groupID)) %>%
      mutate(perm_sex = sex.list.group)

    perm_data <- df %>%
      left_join(bird.list.group %>% select(Color, perm_sex), by = c("actor" = "Color")) %>%
      rename(actor_perm_sex = perm_sex) %>%
      left_join(bird.list.group %>% select(Color, perm_sex), by = c("subject" = "Color")) %>%
      rename(subject_perm_sex = perm_sex)

    perm_data <- perm_data %>%
      mutate(perm_dyad = ifelse(
        actor_perm_sex == subject_perm_sex, "same sex", "opposite sex"
      ))

    perm_summary <- perm_data %>%
      group_by(perm_dyad) %>%
      summarise(mean_events = mean(nevents), .groups = "drop")
  }
}
```

```

    same_sex <- perm_summary$mean_events[perm_summary$perm_dyad == "same sex"]
    opposite_sex <- perm_summary$mean_events[perm_summary$perm_dyad == "opposite sex"]

    perm_results[i, "same sex"] <- ifelse(length(same_sex) == 0, 0, same_sex)
    perm_results[i, "opposite sex"] <- ifelse(length(opposite_sex) == 0, 0, opposite_sex)
  }

par(mfrow=c(1,2))

obs_opp <- observed$mean_events[observed$dyad.sex3 == "opposite sex"]
hist(perm_results[, "opposite sex"], main = "Opposite sex", xlab = "Aggression events")
abline(v = obs_opp, lty = 2, col = "red", lwd = 3)
p_low_opp <- (length(which(perm_results[, "opposite sex"] < obs_opp)) + 1) / (t + 1)
p_low_opp
p_high_opp <- (length(which(perm_results[, "opposite sex"] > obs_opp)) + 1) / (t + 1)
p_high_opp

obs_same <- observed$mean_events[observed$dyad.sex3 == "same sex"]
hist(perm_results[, "same sex"], main = "Same sex", xlab = "Aggression events")
abline(v = obs_same, lty = 2, col = "red", lwd = 3)
p_low_same <- (length(which(perm_results[, "same sex"] < obs_same)) + 1) / (t + 1)
p_low_same
p_high_same <- (length(which(perm_results[, "same sex"] > obs_same)) + 1) / (t + 1)
p_high_same

return(c(p_low_opp = p_low_opp, p_high_opp = p_high_opp, p_low_same = p_low_same, p_high_same = p_high_opp))
}

for (group_name in names(split_dfs_Wrank)) {
  jpeg(file.path(figure_output_folder, paste0("perm_dyad_plot_", group_name, ".jpeg")),
        width = 5, height = 5, units = "in", res = 300)

  this_df <- split_dfs_Wrank[[group_name]]

  pvals <- dyad_perm(this_df, group_name)

  dev.off()

  cat("\nGroup:", group_name, "\n")
  print(pvals)
}

##
## Group: G1
##   p_low_opp  p_high_opp  p_low_same  p_high_same
##   0.1725655  0.8196361   0.7628474   0.2293541

##
## Group: G2
##   p_low_opp  p_high_opp  p_low_same  p_high_same
##   0.02039592 0.97380524 0.95940812 0.03479304

```

```
##
## Group: G3
##   p_low_opp p_high_opp p_low_same p_high_same
## 0.02499500 0.96760648 0.95920816 0.03339332

##
## Group: G4
##   p_low_opp p_high_opp p_low_same p_high_same
## 0.3749250 0.6082783 0.6000800 0.3831234

##
## Group: G5
##   p_low_opp p_high_opp p_low_same p_high_same
## 0.1099780 0.8806239 0.8622276 0.1283743
```

Plot permutation results 2 dyads of five groups

```
set.seed(123)

perm2_dev_df <- data.frame()

sex.list <- c("Female", "Female", "Female", "Female", "Female",
             "Male", "Male", "Male", "Male", "Male")

sex.list <- as.data.frame(sex.list)

for (i in seq_along(split_dfs_Wrank)) {

  this_df <- split_dfs_Wrank[[i]]
  group_name <- unique(this_df$groupID)

  observed <- this_df %>%
    group_by(dyad.sex3) %>%
    summarise(mean_events = mean(nevents), .groups = "drop")

  obs_same <- observed$mean_events[observed$dyad.sex3 == "same sex"]
  obs_opp <- observed$mean_events[observed$dyad.sex3 == "opposite sex"]

  t <- 5000
  perm_results <- matrix(0, nrow = t, ncol = 2)
  colnames(perm_results) <- c("same sex", "opposite sex")

  for (i in 1:t) {
    rows <- sample(nrow(sex.list))
    sex.list.group <- sex.list[rows, ]

    bird.list.group <- bird.list %>%
      filter(combogroupID == unique(this_df$groupID)) %>%
      mutate(perm_sex = sex.list.group)

    perm_data <- this_df %>%
```

```

left_join(bird.list.group %>% select(Color, perm_sex), by = c("actor" = "Color")) %>%
rename(actor_perm_sex = perm_sex) %>%
left_join(bird.list.group %>% select(Color, perm_sex), by = c("subject" = "Color")) %>%
rename(subject_perm_sex = perm_sex) %>%
mutate(perm_dyad = ifelse(actor_perm_sex == subject_perm_sex,
                          "same sex", "opposite sex"))

perm_summary <- perm_data %>%
  group_by(perm_dyad) %>%
  summarise(mean_events = mean(nevents), .groups = "drop")

perm_results[i, "same sex"] <- ifelse("same sex" %in% perm_summary$perm_dyad,
                                     perm_summary$mean_events[perm_summary$perm_dyad=="same sex"],
perm_results[i, "opposite sex"] <- ifelse("opposite sex" %in% perm_summary$perm_dyad,
                                     perm_summary$mean_events[perm_summary$perm_dyad=="opposite sex"],
)

perm2_dev_df <- rbind(perm2_dev_df,
  data.frame(
    group = group_name,
    dyad_type = "same sex",
    deviation = obs_same - perm_results[, "same sex"],
    obs = obs_same,
    perm = perm_results[, "same sex"]
  ),
  data.frame(
    group = group_name,
    dyad_type = "opposite sex",
    deviation = obs_opp - perm_results[, "opposite sex"],
    obs = obs_opp,
    perm = perm_results[, "opposite sex"]
  )
)

perm2_dev_df %>%
  group_by(group, dyad_type) %>%
  summarise(
    obs_val = first(obs),
    prop_lower = mean(first(obs) <= perm)*100,
    prop_higher = mean(first(obs) >= perm)*100,
    mean = mean(perm),
    q75 = quantile(perm, 0.75),
    q25 = quantile(perm, 0.25),
    max = max(perm),
    min = min(perm),
    .groups = "drop"
  )

```

```

## # A tibble: 10 x 10
##   group dyad_type  obs_val prop_lower prop_higher mean  q75  q25  max  min
##   <chr> <chr>      <dbl>    <dbl>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 G1    opposite ~   42.9     82.8     18.0  48.7  51.2  43.5  72.6  27.7
## 2 G1    same sex    55.4     23.7     77.1  49.1  55.2  45.5  85.1  22.8

```


##	3	G2	opposite ~	18.5	98.0	2.62	25.6	29.2	22.6	36.6	17.8
##	4	G2	same sex	34.6	4.06	96.5	25.6	29.2	20.9	37.9	13.2
##	5	G3	opposite ~	10.7	97.5	3.24	17.3	20.7	13.3	25.4	8.74
##	6	G3	same sex	26.6	4.08	96.7	17.4	22.3	13.3	28.3	7.38
##	7	G4	opposite ~	21.1	62.5	39.2	21.9	23.2	20.3	29.1	17.5
##	8	G4	same sex	22.8	40	61.7	22.0	23.9	20.3	27.7	13.7
##	9	G5	opposite ~	6.77	89.0	11.9	7.85	8.37	7.28	10.2	5.8
##	10	G5	same sex	8.91	13.8	87.2	7.88	8.56	7.16	10.7	5.24

```
p_labels <- data.frame(
  group = c("G2", "G3"),
  dyad_type = c("opposite sex", "opposite sex"),
  label = c("p = 0.020", "p = 0.025"),
  y = c(5, 5)
)

facet_labels <- data.frame(
  dyad_type = c("opposite sex", "same sex"),
  label = c("A", "B")
)

dyad2_violin <- ggplot(perm2_dev_df,
  aes(x = factor(group), y = deviation, fill = factor(dyad_type))) +
  geom_violin(alpha = 0.5, width = 1) +
  facet_wrap(~dyad_type, nrow = 1, ncol = 2) +
  scale_fill_manual(values = colrs3) +
  geom_text(
    data = facet_labels, aes(x = -Inf, y = Inf, label = label), inherit.aes = FALSE,
    fontface = "bold", size = 6, hjust = -0.8, vjust = 1) +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed", size = 1) +

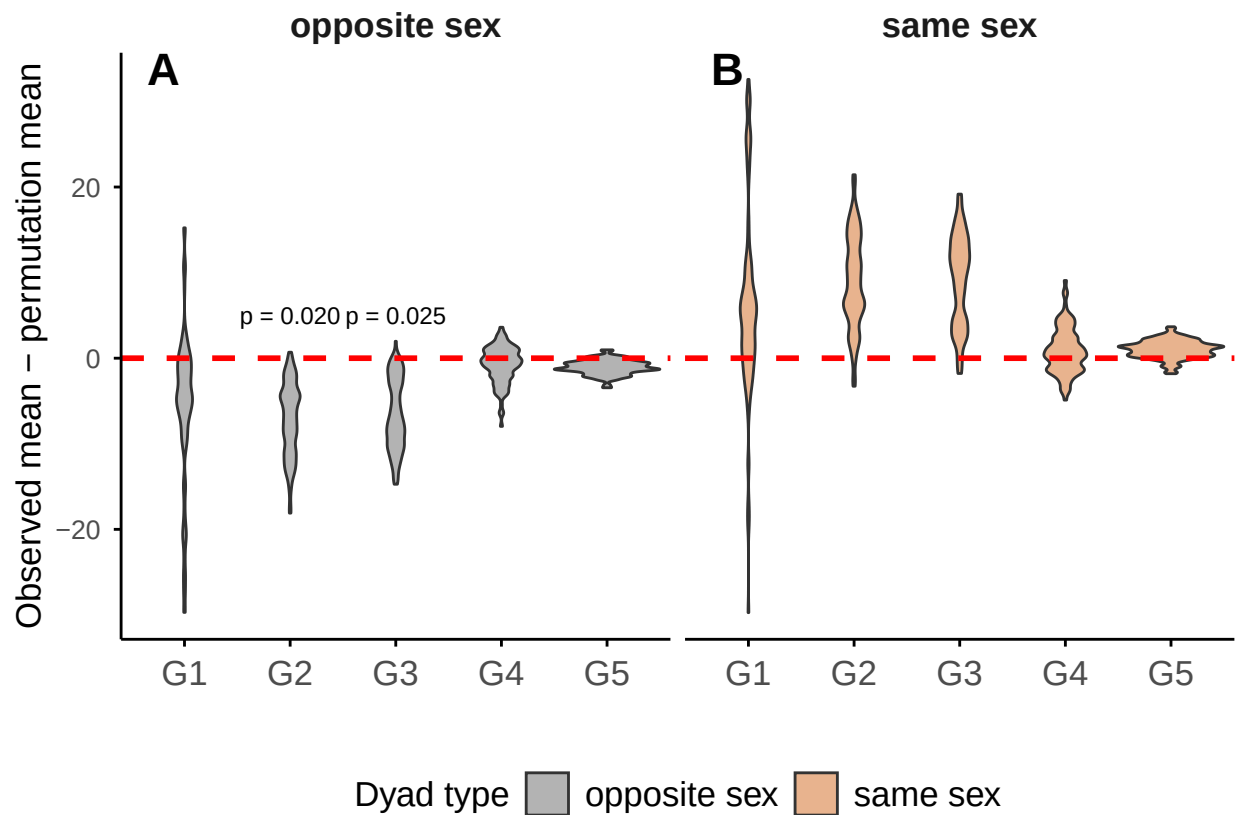
  geom_text(data = p_labels, aes(x = factor(group), y = y, label = label), inherit.aes = FALSE, size = 3) +

  labs(x = "", y = "Observed mean - permutation mean", fill = "Dyad type") +
  theme_minimal() +
  theme(
    axis.title.y = element_text(size = 13),
    axis.text.x = element_text(size = 13),
    axis.text.y = element_text(size = 10),
    legend.text = element_text(size = 13),
    legend.title = element_text(size = 13),
    legend.position = "bottom",
    #plot.margin = margin(10, 50, 10, 50)
  ) +
  scale_x_discrete(
    labels = c("G1", "G2", "G3", "G4", "G5") +
    theme(strip.text = element_text(size = 13, face = "bold")) +
    theme(
      panel.grid.major = element_blank(),
      panel.grid.minor = element_blank()
    ) +
    theme(
      axis.line.y.left = element_line(color = "black"),
      axis.ticks.y = element_line(color = "black"),
```

```
axis.line.x.bottom = element_line(color = "black"),
axis.ticks.x = element_line(color = "black"),
axis.line = element_blank())
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```

```
dyad2_violin
```



```
ggsave(file.path(figure_output_folder, "dyad2_violin.tiff"),
        dyad2_violin,
        device = "tiff",
        width = 18, height = 10, units = "cm")
```

4 dyad types

```
set.seed(123)
dyad_perm <- function(df, group_name){
```

```

observed <- df %>%
  group_by(dyad.sex1) %>%
  summarise(mean_events = mean(nevents), .groups = "drop")

t <- 5000
perm_results <- matrix(0, nrow = t, ncol = 4)
colnames(perm_results) <- c("FF", "FM", "MF", "MM")

for (i in 1:t) {
  rows <- sample(nrow(sex.list))
  sex.list.group <- sex.list[rows, ]

  bird.list.group <- bird.list %>%
    filter(combogroupID == unique(this_df$groupID)) %>%
    mutate(perm_sex = sex.list.group)

  perm_data <- this_df %>%
    left_join(bird.list.group %>% select(Color, perm_sex), by = c("actor" = "Color")) %>%
    rename(actor_perm_sex = perm_sex) %>%
    left_join(bird.list.group %>% select(Color, perm_sex), by = c("subject" = "Color")) %>%
    rename(subject_perm_sex = perm_sex) %>%
    mutate(perm_dyad = paste(actor_perm_sex, subject_perm_sex, sep = "-"))

  perm_summary <- perm_data %>%
    group_by(perm_dyad) %>%
    summarise(mean_events = mean(nevents), .groups = "drop")

  FF <- perm_summary$mean_events[perm_summary$perm_dyad == "Female-Female"]
  FM <- perm_summary$mean_events[perm_summary$perm_dyad == "Female-Male"]
  MF <- perm_summary$mean_events[perm_summary$perm_dyad == "Male-Female"]
  MM <- perm_summary$mean_events[perm_summary$perm_dyad == "Male-Male"]

  perm_results[i, "FF"] <- ifelse(length(FF) == 0, 0, FF)
  perm_results[i, "FM"] <- ifelse(length(FM) == 0, 0, FM)
  perm_results[i, "MF"] <- ifelse(length(MF) == 0, 0, MF)
  perm_results[i, "MM"] <- ifelse(length(MM) == 0, 0, MM)
}

par(mfrow=c(2,2))

obs_MM <- observed$mean_events[observed$dyad.sex1 == "male-male"]
hist(perm_results[, "MM"], main = "MM dyad", xlab = "Aggression events")
abline(v = obs_MM, lty=2, col="red", lwd=3)

#one sided p values on both ends
p_low_MM <- (length(which(perm_results[, "MM"] < obs_MM)) + 1) / (t + 1)
p_low_MM
p_high_MM <- (length(which(perm_results[, "MM"] > obs_MM)) + 1) / (t + 1)
p_high_MM

obs_FF <- observed$mean_events[observed$dyad.sex1 == "female-female"]
hist(perm_results[, "FF"], main = "FF dyad", xlab = "Aggression events")
abline(v = obs_FF, lty=2, col="red", lwd=3)

```

```

p_low_FF <- (length(which(perm_results[, "FF"] < obs_FF)) + 1) / (t + 1)
p_low_FF
p_high_FF <- (length(which(perm_results[, "FF"] > obs_FF)) + 1) / (t + 1)
p_high_FF

obs_FM <- observed$mean_events[observed$dyad.sex1 == "female-male"]
hist(perm_results[, "FM"], main = "FM dyad", xlab = "Aggression events")
abline(v = obs_FM, lty=2, col="red", lwd=3)
p_low_FM <- (length(which(perm_results[, "FM"] < obs_FM)) + 1) / (t + 1)
p_low_FM
p_high_FM <- (length(which(perm_results[, "FM"] > obs_FM)) + 1) / (t + 1)
p_high_FM

obs_MF <- observed$mean_events[observed$dyad.sex1 == "male-female"]
hist(perm_results[, "MF"], main = "MF dyad", xlab = "Aggression events")
abline(v = obs_MF, lty=2, col="red", lwd=3)
p_low_MF <- (length(which(perm_results[, "MF"] < obs_MF)) + 1) / (t + 1)
p_low_MF
p_high_MF <- (length(which(perm_results[, "MF"] > obs_MF)) + 1) / (t + 1)
p_high_MF

#two sided p values
perm_centered_MF <- perm_results[, "MF"] - mean(perm_results[, "MF"])
obs_centered_MF <- obs_MF - mean(perm_results[, "MF"])
p_two_MF <- (length(which(abs(perm_centered_MF) >= abs(obs_centered_MF))) + 1) / (t + 1)

mtext(group_name, side = 3, line = -2, outer = TRUE)

return(c(p_low_MM = p_low_MM, p_high_MM = p_high_MM,
         p_low_FF = p_low_FF, p_high_FF = p_high_FF,
         p_low_FM = p_low_FM, p_high_FM = p_high_FM,
         p_low_MF = p_low_MF, p_high_MF = p_high_MF))
}

for (group_name in names(split_dfs_Wrank)) {
  jpeg(file.path(figure_output_folder, paste0("perm_dyad4_plot_", group_name, ".jpeg")),
       width = 5, height = 5, units = "in", res = 300)

  this_df <- split_dfs_Wrank[[group_name]]

  pvals <- dyad_perm(this_df, group_name)

  dev.off()

  cat("\nGroup:", group_name, "\n")
  print(pvals)
}

```

```

##
## Group: G1
## p_low_MM p_high_MM p_low_FF p_high_FF p_low_FM p_high_FM p_low_MF p_high_MF

```

```
## 0.7246551 0.2707459 0.5776845 0.4177165 0.4283143 0.5670866 0.4815037 0.5138972
```

```
##
```

```
## Group: G2
```

```
## p_low_MM p_high_MM p_low_FF p_high_FF p_low_FM p_high_FM p_low_MF
```

```
## 0.63147371 0.36572685 0.91781644 0.07538492 0.45830834 0.53889222 0.18456309
```

```
## p_high_MF
```

```
## 0.81263747
```

```
##
```

```
## Group: G3
```

```
## p_low_MM p_high_MM p_low_FF p_high_FF p_low_FM p_high_FM p_low_MF
```

```
## 0.93681264 0.05938812 0.53089382 0.45210958 0.20895821 0.78384323 0.33333333
```

```
## p_high_MF
```

```
## 0.66286743
```

```
##
```

```
## Group: G4
```

```
## p_low_MM p_high_MM p_low_FF p_high_FF p_low_FM p_high_FM p_low_MF p_high_MF
```

```
## 0.8422316 0.1531694 0.2181564 0.7772446 0.4183163 0.5770846 0.5006999 0.4947011
```

```
##
```

```
## Group: G5
```

```
## p_low_MM p_high_MM p_low_FF p_high_FF p_low_FM p_high_FM p_low_MF
```

```
## 0.92441512 0.06978604 0.13317337 0.86102779 0.27554489 0.71865627 0.61727654
```

```
## p_high_MF
```

```
## 0.37692462
```

Plot permutation results 4 dyads of five groups

```
perm4_dev_df <- data.frame()

for (i in seq_along(split_dfs_Wrank)) {

  this_df <- split_dfs_Wrank[[i]]
  group_name <- unique(this_df$groupID)

  observed <- this_df %>%
    group_by(dyad.sex1) %>%
    summarise(mean_events = mean(nevents), .groups = "drop")

  obs_MM <- observed$mean_events[observed$dyad.sex1 == "male-male"]
  obs_FF <- observed$mean_events[observed$dyad.sex1 == "female-female"]
  obs_FM <- observed$mean_events[observed$dyad.sex1 == "female-male"]
  obs_MF <- observed$mean_events[observed$dyad.sex1 == "male-female"]

  t <- 5000
  perm_results <- matrix(0, nrow = t, ncol = 4)
  colnames(perm_results) <- c("FF", "FM", "MF", "MM")
}
```

```

for (i in 1:t) {
  rows <- sample(nrow(sex.list))
  sex.list.group <- sex.list[rows, ]

  bird.list.group <- bird.list %>%
    filter(combogroupID == unique(this_df$groupID)) %>%
    mutate(perm_sex = sex.list.group)

  perm_data <- this_df %>%
    left_join(bird.list.group %>% select(Color, perm_sex), by = c("actor" = "Color")) %>%
    rename(actor_perm_sex = perm_sex) %>%
    left_join(bird.list.group %>% select(Color, perm_sex), by = c("subject" = "Color")) %>%
    rename(subject_perm_sex = perm_sex) %>%
    mutate(perm_dyad = paste(actor_perm_sex, subject_perm_sex, sep = "-"))

  perm_summary <- perm_data %>%
    group_by(perm_dyad) %>%
    summarise(mean_events = mean(nevents), .groups = "drop")

  FF <- perm_summary$mean_events[perm_summary$perm_dyad == "Female-Female"]
  FM <- perm_summary$mean_events[perm_summary$perm_dyad == "Female-Male"]
  MF <- perm_summary$mean_events[perm_summary$perm_dyad == "Male-Female"]
  MM <- perm_summary$mean_events[perm_summary$perm_dyad == "Male-Male"]

  perm_results[i, "FF"] <- ifelse(length(FF) == 0, 0, FF)
  perm_results[i, "FM"] <- ifelse(length(FM) == 0, 0, FM)
  perm_results[i, "MF"] <- ifelse(length(MF) == 0, 0, MF)
  perm_results[i, "MM"] <- ifelse(length(MM) == 0, 0, MM)
}

perm4_dev_df <- rbind(perm4_dev_df,
  data.frame(
    group = group_name,
    dyad_type = "FF",
    obs = obs_FF,
    deviation = obs_FF - perm_results[, "FF"],
    perm = perm_results[, "FF"]
  ),
  data.frame(
    group = group_name,
    dyad_type = "FM",
    obs = obs_FM,
    deviation = obs_FM - perm_results[, "FM"],
    perm = perm_results[, "FM"]
  ),
  data.frame(
    group = group_name,
    dyad_type = "MM",
    obs = obs_MM,
    deviation = obs_MM - perm_results[, "MM"],
    perm = perm_results[, "MM"]
  ),

```

```

      data.frame(
        group = group_name,
        dyad_type = "MF",
        obs = obs_MF,
        deviation = obs_MF - perm_results[, "MF"],
        perm = perm_results[, "MF"]
      ))
}

perm4_dev_df %>%
  group_by(group, dyad_type) %>%
  summarise(
    obs_val = first(obs),
    prop_lower = mean(perm < first(obs))*100,
    prop_higher = mean(perm > first(obs))*100,
    mean = mean(perm),
    q75 = quantile(perm, 0.75),
    q25 = quantile(perm, 0.25),
    max = max(perm),
    min = min(perm),
    .groups = "drop"
  )

```

```

## # A tibble: 20 x 10
##   group dyad_type obs_val prop_lower prop_higher mean q75 q25 max min
##   <chr> <chr>      <dbl>      <dbl>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 G1     FF         51.6        57.4        42.1  49.4  60.9  33.3 127. 18.2
## 2 G1     FM         42.3        42.1        57.4  47.1  56.4   34  103   1
## 3 G1     MF         43.6        48.4        51.2  47.3  56.4  34.6 103   1
## 4 G1     MM         59.2        73.0        26.5  49.6  60.8  33.5 127. 18.2
## 5 G2     FF         43          91.4         7.76  25.5  31.5   18  59.4  8.75
## 6 G2     FM         21.9        45.9        53.7  24.5  31.4  16.1  51.8  1.43
## 7 G2     MF         15.3        19.5        80.1  24.4  31.3  16.1  51.8  1.43
## 8 G2     MM         26.2        61.5        38.1  25.7  31.9  18.2  59.4  8.75
## 9 G3     FF         16.5        52.8        45.7  17.3   24   9.69  42.2  2.25
## 10 G3    FM          9.67        22.2        76.9  16.4  21.7   10  33.8  3.6
## 11 G3    MF         11.5        33.8        65.6  16.6  22.8   10  33.8  3.6
## 12 G3    MM         33.2        93.3         6.18  17.1  23.9   9.55  42.2  2.25
## 13 G4     FF         16.7        22.9        76.6  21.9  25.9  16.9  38.8  6.17
## 14 G4     FM         20.3        43.0        56.5  21.1  25.6  17.4  35.9   1
## 15 G4     MF         22.1        49.2        50.3  21.4  25.7  17.7  35.9   1
## 16 G4     MM         28.9        83.8        15.6  22.1  26.6   17  38.8  6.17
## 17 G5     FF          4.67        13.4        86.3   7.74  9.36  5.82  13.8  2.22
## 18 G5     FM          5.54        27.6        72.1   7.42  9.5   5.31  13.1   1
## 19 G5     MF          8.56        62.3        37.3   7.38  9.5    5   13.1   1
## 20 G5     MM         11.8        93.1         6.54  7.71  9.36  5.7   13.8  2.22

```

```

colrs2 <- c("FM"="#a6611a",
            "MF"="#dfc27d",
            "FF"="#80cdc1",
            "MM"="#018571")

```

```
perm4_dev_df %>%
```

```

group_by(group) %>%
  filter(dyad_type=="FF") %>%
  summarise(max = max(deviation),
            min = min(deviation))

## # A tibble: 5 x 3
##   group   max   min
##   <chr> <dbl> <dbl>
## 1 G1    33.4 -75.6
## 2 G2    34.2 -16.4
## 3 G3    14.2 -25.7
## 4 G4    10.6 -22.1
## 5 G5     2.44 -9.08

facet_labels4 <- data.frame(
  dyad_type = c("female-male", "male-female", "female-female", "male-male"),
  label = c("A", "B", "C", "D")
)

perm4_dev_df <- perm4_dev_df %>%
  mutate(
    dyad_type = case_when(
      dyad_type == "FM" ~ "female-male",
      dyad_type == "MF" ~ "male-female",
      dyad_type == "FF" ~ "female-female",
      dyad_type == "MM" ~ "male-male"
    )
  )

colrs4 <- c("female-male"="#a6611a",
            "male-female"="#dfc27d",
            "female-female"="#80cdc1",
            "male-male"="#018571")

#p_labels4 <- data.frame(
# group = c("G3", "G5"),
# dyad_type = c("male-male", "male-male"),
# label = c("p = 0.059", "p = 0.070"),
# y = c(-15, -15) )

dyad4_violin <- ggplot(perm4_dev_df,
  aes(x = factor(group), y = deviation, fill = factor(dyad_type))) +
  geom_violin(alpha = 0.5, width = 1) +
  facet_wrap(~factor(dyad_type, c("female-male", "male-female", "female-female", "male-male")), nrow = 2) +
  theme_minimal() +
  scale_fill_manual(values = colrs4) +
  geom_text(
    data = facet_labels4, aes(x = -Inf, y = Inf, label = label), inherit.aes = FALSE,
    fontface = "bold", size = 6, hjust = -0.8, vjust = 1) +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed", size = 1) +
  # geom_text(data = p_labels4, aes(x = factor(group), y = y, label = label), inherit.aes = FALSE, size = 6) +
  labs(x = "", y = "Observed mean - permutation mean", fill = "Dyad type") +
  theme_minimal() +

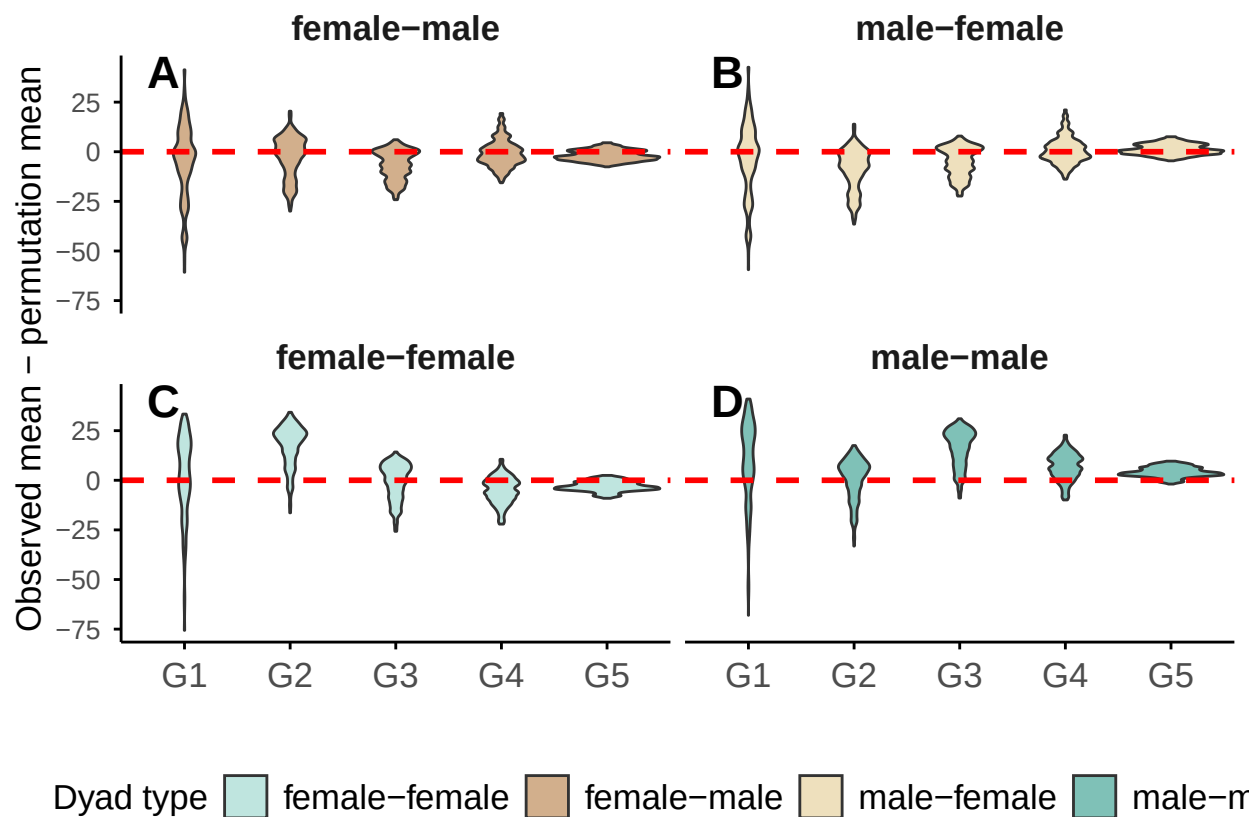
```



```

theme(
  axis.title.y = element_text(size = 13),
  axis.text.x = element_text(size = 13),
  axis.text.y = element_text(size = 10),
  legend.text = element_text(size = 13),
  legend.title = element_text(size = 13),
  legend.position = "bottom"
)+
scale_x_discrete(
  labels = c("G1", "G2", "G3", "G4", "G5" ) +
  theme(strip.text = element_text(size = 13, face = "bold")) +
  theme(
    panel.grid.major = element_blank() ,
    panel.grid.minor = element_blank()
  )+
  theme(
    axis.line.y.left = element_line(color = "black"),
    axis.ticks.y = element_line(color = "black"),
    axis.line.x.bottom = element_line(color = "black"),
    axis.ticks.x = element_line(color = "black"),
    axis.line = element_blank()
  )
)
dyad4_violin

```



```
ggsave(file.path(figure_output_folder, "dyad4_violin.tiff"),
       dyad4_violin,
       device = "tiff",
       width = 18, height = 13, units = "cm")
```

#3. Group level

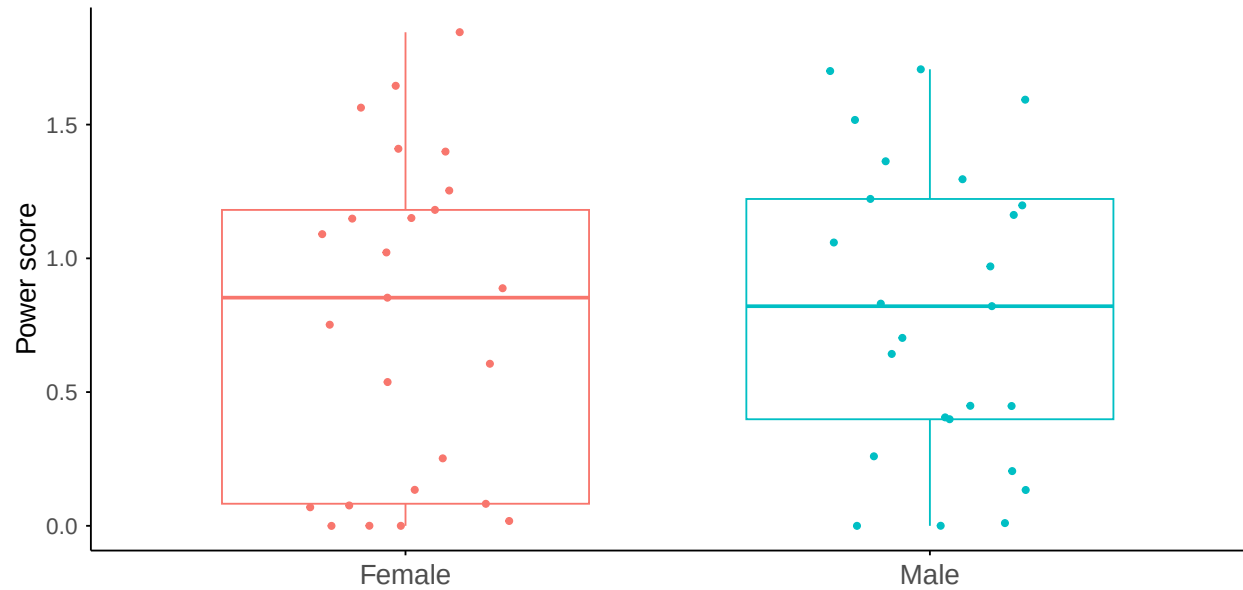
##Plot rank vs. sex - all five group together

```
summary(rank_df_all$power)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0000  0.2164  0.8258  0.7814  1.2159  1.8452
```

```
power_sex_model_boxplot_all <-
  ggboxplot(rank_df_all, x = "sex", y = "power", color = "sex", add = "jitter") +
  scale_color_manual(values = colrs) +
  labs(y = "Power score", x = "") +
  theme_minimal() +
  theme(legend.position="none") +
  theme(
    axis.title.y = element_text(size = 16),
    axis.text.y = element_text(size = 13),
    axis.text.x = element_text(size = 16)
  ) +
  theme(
    panel.grid.major = element_blank() ,
    panel.grid.minor = element_blank()
  ) +
  theme(
    axis.line.y.left = element_line(color = "black"),
    axis.line.x.bottom = element_line(color = "black"),
    axis.ticks = element_line(color = "black"),
    axis.line = element_blank()
  )

power_sex_model_boxplot_all
```



```
ggsave(file.path(figure_output_folder, "power_sex_model_boxplot_all.tiff"),
        power_sex_model_boxplot_all,
        device = "tiff",
        width = 10, height = 10, units = "cm")
```

```
## Analysis LM - 5 groups all together
```

```
power_sex_model_all <- lmer(power ~ sex + (1 | groupID), data = rank_df_all)
```

```
## boundary (singular) fit: see help('isSingular')
```

```
summary(power_sex_model_all)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: power ~ sex + (1 | groupID)
## Data: rank_df_all
##
## REML criterion at convergence: 90.7
##
## Scaled residuals:
##   Min       1Q   Median       3Q      Max
## -1.380 -1.006  0.038  0.723  1.865
##
## Random effects:
##   Groups   Name      Variance Std.Dev.
##   groupID  (Intercept) 0.000    0.0000
##   Residual                0.339    0.5822
## Number of obs: 50, groups: groupID, 5
##
## Fixed effects:
##              Estimate Std. Error t value
```

```
## (Intercept) 0.75918 0.11645 6.52
## sexMale 0.04453 0.16468 0.27
##
## Correlation of Fixed Effects:
## (Intr)
## sexMale -0.707
## optimizer (nloptwrap) convergence code: 0 (OK)
## boundary (singular) fit: see help('isSingular')
```

#groupID does not explain variance, and thus simplified model

```
power_sex_model_all <- lm(power ~ sex, data = rank_df_all)
summary(power_sex_model_all)
```

```
##
## Call:
## lm(formula = power ~ sex, data = rank_df_all)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.80372 -0.58544  0.02213  0.42097  1.08606
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.75918    0.11645    6.52 4.01e-08 ***
## sexMale      0.04453    0.16468    0.27  0.788
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5822 on 48 degrees of freedom
## Multiple R-squared:  0.001521, Adjusted R-squared: -0.01928
## F-statistic: 0.07313 on 1 and 48 DF, p-value: 0.788
```

```
r2(power_sex_model_all)
```

```
## # R2 for Linear Regression
##      R2: 0.002
##   adj. R2: -0.019
```

```
performance::check_singularity(power_sex_model_all)
```

```
## [1] FALSE
```

```
power_sex_model_all_summary <- capture.output(summary(power_sex_model_all))
writeLines(power_sex_model_all_summary, file.path(table_output_folder, "power_sex_model_all_summary.txt"))

doc <- read_docx()
for(line in power_sex_model_all_summary){doc <- body_add_par(doc, line, style = "Normal")}
print(doc, target = file.path(table_output_folder, "power_sex_model_all_summary.docx"))

summary(power_sex_model_all)$coefficients %>%
```

```

as.data.frame() %>%
tibble::rownames_to_column("Predictor") %>%
flextable::flextable() %>%
flextable::colformat_double(digits = 3) %>%
flextable::add_footer_lines(
  "Estimate = coefficient; Std. Error = standard error; z value = t statistic; Pr(>|z|) = p-value."
) %>%
flextable::autofit() %>%
flextable::save_as_docx(
  path = file.path(table_output_folder, "power_sex_model_all_estimates.docx")
)

```

#check the model

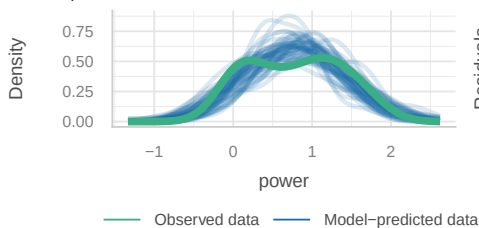
```

power_sex_model_all_diagnostics_plot <- check_model(power_sex_model_all, title_size = 8, base_size = 7)
power_sex_model_all_diagnostics_plot

```

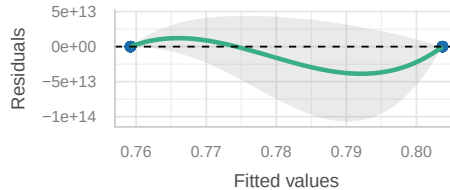
Posterior Predictive Check

Model-predicted lines should resemble observed data line



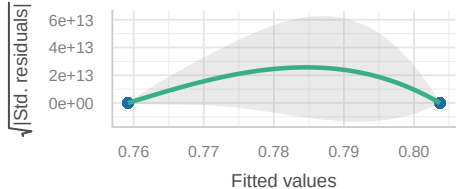
Linearity

Reference line should be flat and horizontal



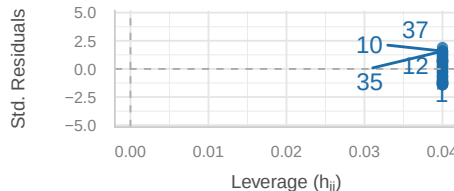
Homogeneity of Variance

Reference line should be flat and horizontal



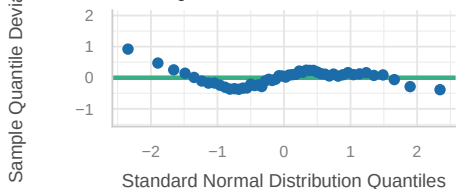
Influential Observations

Points should be inside the contour lines



Normality of Residuals

Points should fall along the line



saving the plot

```

tiff(file = file.path(figure_output_folder, "power_sex_model_all_diagnostics.tiff"),
     width = 18, height = 18, units = "cm", res = 300)

```

```

print(power_sex_model_all_diagnostics_plot)

```

```

dev.off()

```

```
## pdf
## 2
```

LM - 5 groups separately

```
shapiro.test(rank_df_all$power) #Normality violation
```

```
##
## Shapiro-Wilk normality test
##
## data: rank_df_all$power
## W = 0.92888, p-value = 0.005015
```

```
bartlett.test(power ~ groupID, data = rank_df_all)
```

```
##
## Bartlett test of homogeneity of variances
##
## data: power by groupID
## Bartlett's K-squared = 0.6937, df = 4, p-value = 0.9521
```

```
power_sex_lm <- function(df) {

df1 <- df %>%
  select(birdID = actor, rank = actor.rank, power = actor.power)

df2 <- df %>%
  select(birdID = subject, rank = subject.rank, power = subject.power)

rank_df <- bind_rows(df1, df2) %>%
  distinct(birdID, rank, power) %>%
  left_join(bird.list %>% select(Color, Sex), by = c("birdID" = "Color"))

power_sex_model <- lm(power ~ Sex, data = rank_df)
check <- check_model(power_sex_model)
print(check)
return(power_sex_model)
}

model_summaries <- list()

for (group_name in names(split_dfs_Wrank)) {
  this_df <- split_dfs_Wrank[[group_name]]
  model <- power_sex_lm(this_df)
  model_summaries[[group_name]] <- tbl_regression(model, pvalue_fun = ~ style_pvalue(.x, digits = 2)) %>%
    bold_labels() %>%
    italicize_levels() %>%
    remove_row_type(variables = everything(), type = "reference", level_value = NULL) %>%
    modify_table_body(~ .x %>%
      mutate(
        label = case_when(
          label == "Male" ~ "Male (vs. female)",
```

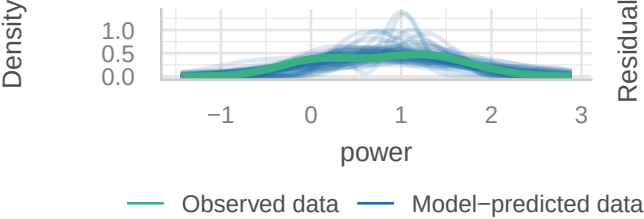
```

    TRUE ~ label)))
}

```

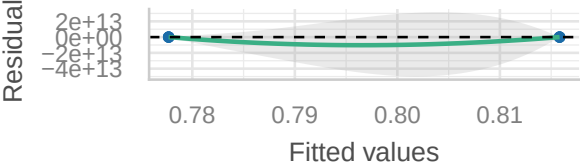
Posterior Predictive Check

Model-predicted lines should resemble observed data



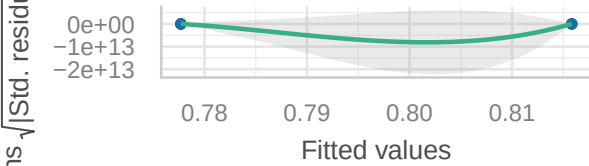
Linearity

Reference line should be flat and horizontal



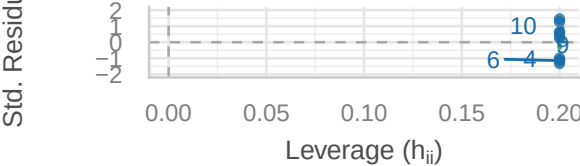
Homogeneity of Variance

Reference line should be flat and horizontal



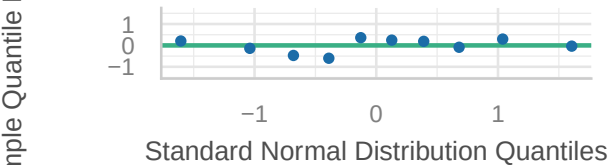
Influential Observations

Points should be inside the contour lines



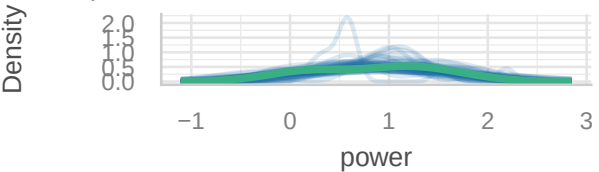
Normality of Residuals

Points should fall along the line



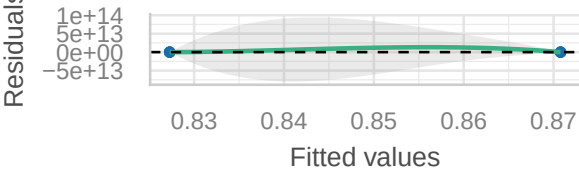
Posterior Predictive Check

Model-predicted lines should resemble observed data



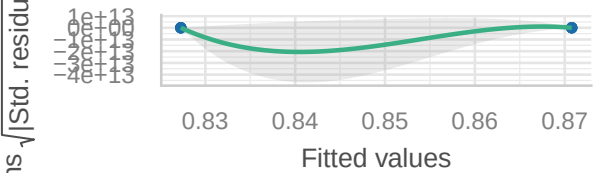
Linearity

Reference line should be flat and horizontal



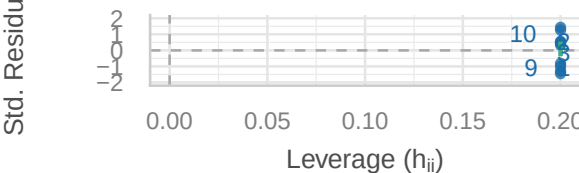
Homogeneity of Variance

Reference line should be flat and horizontal



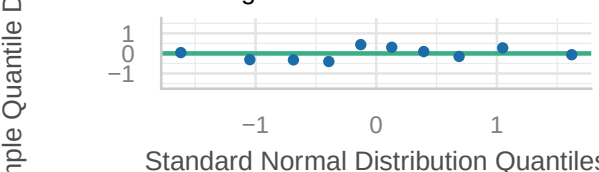
Influential Observations

Points should be inside the contour lines



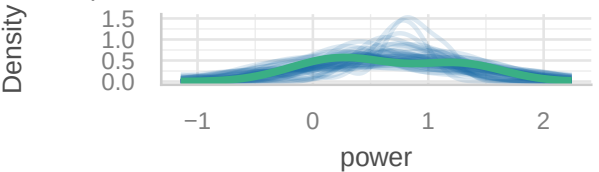
Normality of Residuals

Points should fall along the line



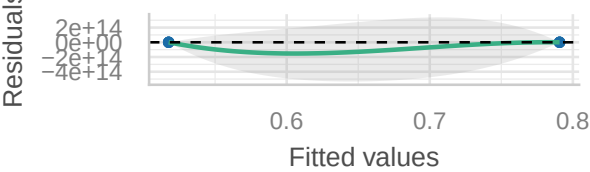
Posterior Predictive Check

Model-predicted lines should resemble observed data



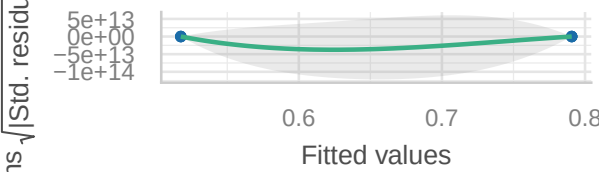
Linearity

Reference line should be flat and horizontal



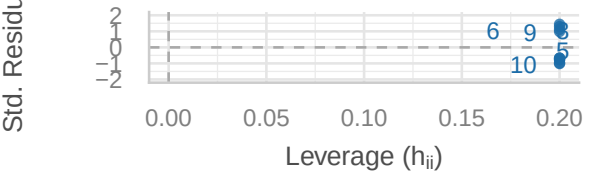
Homogeneity of Variance

Reference line should be flat and horizontal



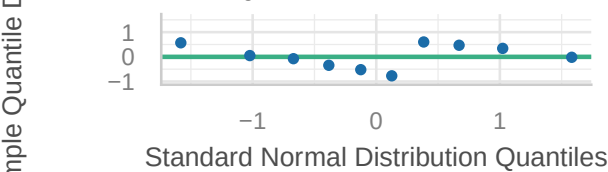
Influential Observations

Points should be inside the contour lines



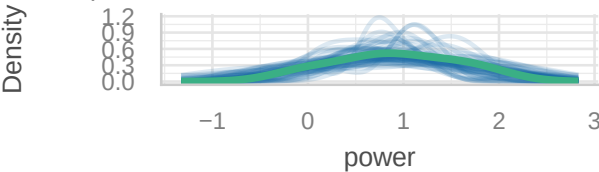
Normality of Residuals

Points should fall along the line



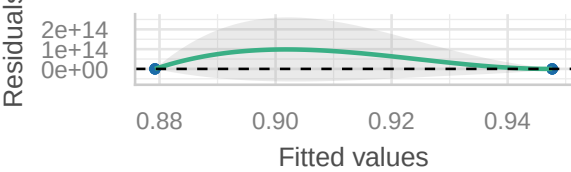
Posterior Predictive Check

Model-predicted lines should resemble observed data



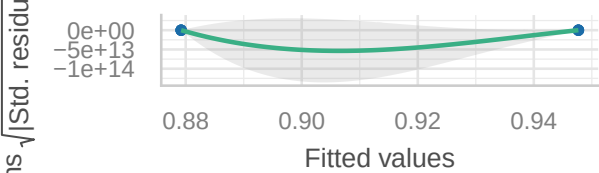
Linearity

Reference line should be flat and horizontal



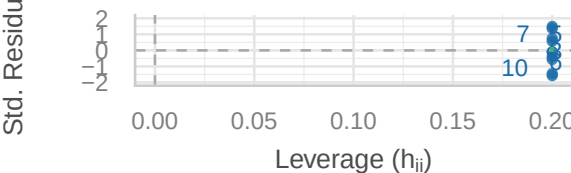
Homogeneity of Variance

Reference line should be flat and horizontal



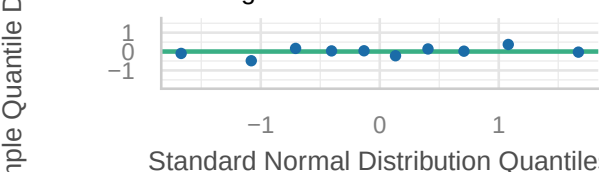
Influential Observations

Points should be inside the contour lines



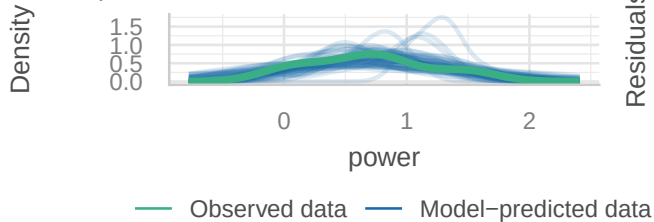
Normality of Residuals

Points should fall along the line



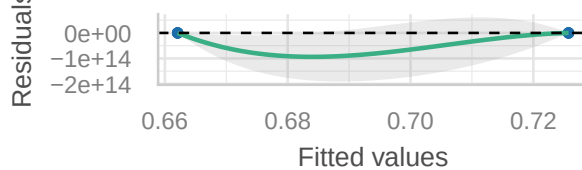
Posterior Predictive Check

Model-predicted lines should resemble observed data



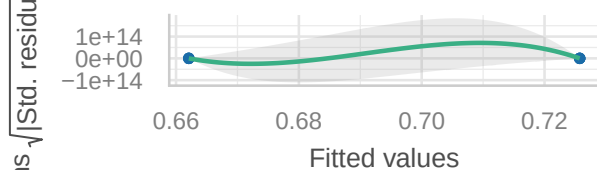
Linearity

Reference line should be flat and horizontal



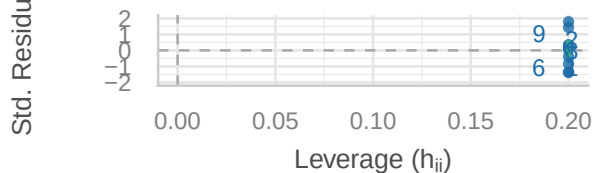
Homogeneity of Variance

Reference line should be flat and horizontal



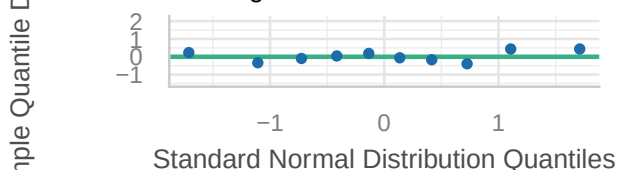
Influential Observations

Points should be inside the contour lines



Normality of Residuals

Points should fall along the line



```
combined_df <- imap_dfr(
  model_summaries,
  function(tbl, group_name) {

    tbl$table_body %>%
      filter(label == "Male (vs. female)") %>% # keep only effect row
      mutate(Group = group_name, .before = 1)
  }
)

combined_df <- combined_df %>%
  mutate(
    Group = paste0("G", match(Group, names(model_summaries)))
  )

combined_df <- combined_df %>%
  select(Group, estimate, conf.low, conf.high, p.value) %>%
  mutate(across(where(is.numeric), ~ signif(.x, 2)))

combined_df
```

```
## # A tibble: 5 x 5
##   Group estimate conf.low conf.high p.value
##   <chr>      <dbl>    <dbl>    <dbl>    <dbl>
## 1 G1        0.038    -0.97      1      0.93
```

## 2 G2	0.044	-0.89	0.97	0.92
## 3 G3	0.27	-0.56	1.1	0.47
## 4 G4	-0.068	-1.1	0.91	0.88
## 5 G5	-0.064	-0.85	0.72	0.86

```
flextable(combined_df) %>%
  save_as_docx( path = paste0(table_output_folder, "/power_sex_model_group_summary.docx"))
```

Plots of continous aggression events and ordinal social rankings

```
names(split_dfs_Wrank) <- c("G1", "G2", "G3", "G4", "G5")

plot_group_hierarchy <- function(df, group_name) {

ntotal <- df %>% summarise(ntotal = sum(nevents))

el <- df %>%
  group_by(actor, subject, actor.sex, subject.sex, power.diff, rank.diff, dyad.sex3) %>%
  mutate(prop.agg=nevents/ntotal$ntotal) %>% #number and proportion of aggression events by dyad type
  select(actor, subject, actor.sex, subject.sex, power.diff, rank.diff, dyad.sex3, nevents, prop.agg)

df1 <- df %>%
  select(birdID = actor, rank = actor.rank, power = actor.power)

df2 <- df %>%
  select(birdID = subject, rank = subject.rank, power = subject.power)

rank_df <- bind_rows(df1, df2) %>%
  distinct(birdID, rank, power)

atts <- inner_join(el, rank_df, join_by("actor" == "birdID"))

atnet <- graph_from_data_frame(
  el,
  directed = TRUE
)

V(atnet)$power <- atts$power[match(V(atnet)$name, atts$actor)]
V(atnet)$rank <- atts$rank[match(V(atnet)$name, atts$actor)]
V(atnet)$sex <- atts$actor.sex[match(V(atnet)$name, atts$actor)]

# Add conditional color, based on direction of aggression
E(atnet)[dyad.sex3=="same sex"]$dcolor <- adjustcolor("chocolate", alpha = 0.4)
E(atnet)[dyad.sex3=="opposite sex"]$dcolor <- adjustcolor("dimgray", alpha = 0.4)
E(atnet)$edge.curved <- ifelse(E(atnet)$dyad.sex3 == "same sex",
                             0.2, # curve above
                             -0.2) # curve below

# Set custom attribute-ordered network layout
y <- V(atnet)$power
x <- 1
```

```

birdID <- V(atnet)$name
attord<- cbind.data.frame(birdID, x, y) #attribute-ordered layout of nodes

# Add node color pattern by sex
colrs <- c("#F8766D", "#00BFC4")
names(colrs) <- c("Female", "Male")
V(atnet)$color <- colrs[V(atnet)$sex]

#Add text box
pad_x <- 0.03
pad_y <- 0.005

par(mar = c(2, 4.5, 2, 0))
#par(mar = c(1.5,6,2,0)) # tighten margins

plot(NULL,
      #xlim=c(1,1),
      xlim=c(0.8, 1.2),
      ylim=c(0,2),
      ylab="Power score", xlab="",
      cex.axis=1,
      cex.lab=2,
      axes = FALSE,
      xaxt = "n")
#box()
axis(1, at = 1, cex.axis=1, label = FALSE)
axis(2, at = seq(0,2, 0.25),las=1, cex.axis=1.3)
abline(v = 1, lwd=1.5, col = "black")

# Set node size, label size, tie width
nodesize<-4 #set node size
edge.multiplier <- 100
edge.minimum <- 0.25
tiewidth <- (E(atnet)$prop.agg*edge.multiplier)+edge.minimum #set edge width
labelsize<-0.5 #set node label size

#plot network
plot.igraph(atnet,
add=TRUE,
rescale=FALSE, asp = 0,
layout=attord[,2:3],#sets as attribute-ordered layout, defined above
vertex.size=nodesize, #this references the node circle size, set above
vertex.label = "", #sets node label colors (ids) to black
vertex.color=V(atnet)$color, #sets node fill color by sex
edge.width=tiewidth, #this references the edge width, set above
edge.arrow.size=0.01, #this makes the arrows on the ends of the edges very small
edge.curved=E(atnet)$edge.curved,
edge.loop.angle=0,
edge.color= E(atnet)$dcolor, #this sets the edge color, based on the colors set above
axes=TRUE,
#xlim=c(1,1),
xlim=c(0.8, 1.2),
ylim=c(-0.25,2.5)

```

```

)
text(x=1.05, y=0, label=paste0("N = ", ntotal$ntotal), adj = c(0,0), font = 3, cex=2, pch = 100)
title(main = group_name, cex.main = 3)

}

tiff(file.path(figure_output_folder, "hierarchy_all_groups.tiff"),
      width=25, height=25, units="cm", res=300)

ncol <- 3
nrow <- 2

par(mfrow = c(nrow, ncol),
    mar = c(2, 2, 2, 1),
    oma = c(2, 2, 2, 2))

for (group_name in names(split_dfs_Wrank)) {
  this_df <- split_dfs_Wrank[[group_name]]
  plot_group_hierarchy(this_df, group_name)
}

plot.new()

legend(x = 0, y = 0.7, legend = c("Female", "Male"), pch = 21, pt.bg = c("#F8766D", "#00BFC4"), title =
legend(x = 0, y = 0.4, legend = c("same sex", "opposite sex"), col = c("chocolate", "dimgray"), lwd = 2,
dev.off()

## pdf
## 2

```